GroundGate: Pre-Execution Grounding of Agent Tool Calls Under Imperfect Evidence

Perceptron Hackathon 2026 · Round 1 · Theme 6 (AI You Can Actually Trust) · Problem 9: Catching an Agent Before It Acts on a Bad Idea

Team: [team name] · Team ID: [ID] · Leader: [name] · Members: [name 2], [name 3] · Institution: [institution]

1. Problem and Theme Chosen

A tool-calling agent can emit a well-formed, permitted action resting on false facts, such as a refund for an order that does not exist, belongs to another customer, or was already refunded. The failure is silent. Permission layers (AgentSpec, Progent, CaMeL) ask whether the agent may act, not whether its assumed facts are true.

2. Understanding of the Challenge

The hard part is that real evidence is imperfect:

- **Absence is not contradiction.** A cache lacking an order has not shown it is fabricated. Blocking on absence causes false blocks; trusting stale entries causes wrong executions.
- **Verification costs.** Each authoritative lookup has a latency and budget cost, so the gate must decide when to pay it.
- **Check-to-execute gap.** A verdict can be outdated by mutation time; retries and concurrent requests must not act twice.

3. Proposed Approach

Testbed: three mutating tools (refund, cancel, change address), a SQLite system of record and an incomplete, possibly stale cache. The gate is standard-library Python; a LangGraph agent will call it.

1. **Typed action record:** validated arguments, reversibility, amount, idempotency key, and premises derived from the tool schema.
2. **Evidence resolution:** observations carry source, authority and freshness. Authoritative beats cache; stale entries never support irreversible actions; free text is never evidence. Verdicts are SUPPORTED, CONTRADICTED or UNVERIFIABLE.
3. **Verification cascade:** exact checks, the cache, then at most one read-only authoritative lookup under a budget, timeout and circuit breaker.
4. **Risk-table policy:** CONTRADICTED blocks, SUPPORTED allows. UNVERIFIABLE escalates if irreversible or high-value, and is allowed with logging otherwise. Rule table with assumed costs, not a calibrated model.
5. **Execution barrier:** the executor accepts only `(tool, args, request_id)`, re-derives premises, risk and idempotency key, requires an expiring single-use action-bound token, then rechecks premises and applies a version-conditional update in one `BEGIN IMMEDIATE` transaction.

4. Minimum Viable Implementation

(1) A checker verifying refund calls against a source before execution; (2) a demo stopping a fabricated order ID, another customer's order, a forged token and a hostile payload. It needs no LLM or GPU. Already built: 81 passing automated tests (unit, security, race, benchmark-integrity), a benchmark that reruns from a clean clone, and a results verifier reporting 0 discrepancies. Not yet built: the LangGraph agent and any real-model run.

5. Stretch Goals

Risk-based routing is implemented as rules: reversibility and amount choose between cache-only, one lookup, or escalation. A 60–100 case red-team suite is planned. A fast/careful model router (LLM judge as careful tier) is planned, not built.

6. Novelty and Differentiation

We claim no first on any component. Deterministic gates over state exist (Reason Less, Verify More, arXiv 2607.07405; LedgerAgent, 2606.20529; Zwerdling et al., 2507.16459; NOMOS, 2610.11030). Budgeted verification under uncertainty appears in SafeCommit (2608.04289) and BAVAR (simulated). Our review found no controlled study of business-API gating that separates four state views (agent-visible, verification-time, verifier-visible, mutation-time) while degrading the evidence source. Our contribution is that experiment and its findings, not a new algorithm.

7. Evaluation Plan and Current Results

Design. 20 synthetic worlds × 500 actions × 8 source conditions (cache gaps, staleness, replica lag, outages, lookup budgets, drift); parameters frozen on separate dev seeds; four held-out error families; harm labelled from mutation-time state. Baselines: no guard, exact rules, closed-world cache, a fact gate reimplemented from NOMOS's described behaviour, always-query, GroundGate and three ablations. Metrics: harmful actions passing the gate, false blocks, escalations, valid completions, lookups per action, expected loss; 95% bootstrap intervals over worlds.

Findings so far (synthetic, not real model errors).

- Cache-first routing used 0.08–0.22 fewer lookups per action than always-query when the source was available, at equal gate-level safety.
- With scarce lookups it completed 10.6 more points of valid actions (CI 9.8–11.4) and escalated 20.5 points fewer.
- Gate-level safety is bounded by evidence quality: harmful pass rates were 2.5% (lagging replica), 10.3% (scarce lookups), 16.0% (all degradations). Commit-time recheck stopped all, but it reads the state that defines the label, so this is circular.

- The cheapest policy depends on assumed costs: at a false-block cost of 1 a closed-world guard often wins; at 20 GroundGate wins except in the ideal condition.
- In 30 multi-process races, naive check-then-act violated the refund invariant in all 30; the protected executor in none (95% upper bound 11%).

Not done: real model errors, human-written cases, an LLM-judge baseline, a lagging executor view. Always-query is as safe as GroundGate; the closed-world guard completes only about 60% of valid actions.

8. Feasibility, Team Fit and Risks

Skills: [member 1: agent frameworks], [member 2: evaluation], [member 3: backend and security]. Plan: to 9 Nov LangGraph agent; 10–20 Nov MVP demo; 21–30 Nov elicit real model errors, midway report; 1–14 Dec LLM judge, red-team suite, rehearsal. Risks: real model errors may be rare (reported separately); synthetic circularity (stated); cost-dependent ranking; scope growth (stretch items drop first).

References

Reddy et al., arXiv:2607.07405. Yu et al., NOMOS, arXiv:2610.11030. Zwerdling et al., arXiv:2507.16459. LedgerAgent, arXiv:2606.20529. SafeCommit, arXiv:2608.04289. BAVAR, preprints.org 202608.2057. AgentSpec, arXiv:2503.18666. Progent, arXiv:2504.11703. CaMeL, arXiv:2503.18813.